7장 임시 처리

지금까지 커밋과 브랜치를 이용하여 변경된 코드를 기록하는 방법을 학습했습니다. 코드를 생성하고 수정하는 모든 작업은 워킹 디렉터리에서 수행합니다. 워킹 디렉터리는 스테이지 영역을 통해 커밋합니다. 안정적으로 깃을 활용하려면 워킹 디렉터리와 스테이지 영역을 관리해야 합니다. 이와 관련 있는 몇 가지 기능을 알아보겠습니다.

7.1 스태시

7.2 워킹 디렉터리 청소

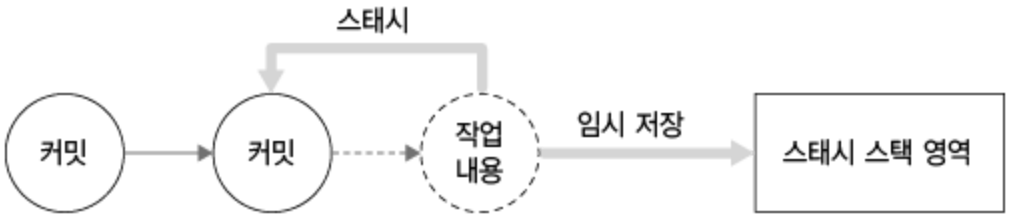
7.3 정리

7.1 스태시

작업 브랜치를 변경하려면 워킹 디렉터리는 깨끗한(clean) 상태로 정리되어 있어야 합니다. 워킹 디렉터리에 작업 중인 내용이나 커밋되지 않은 변경 사항들이 남아 있으면 브랜치를 변경할 수 없습니다. 예를 들어 브랜치에서 코드를 수정하는 도중에 새로운 버그가 발견됩니다. 또는 고객의 긴급한 요청으로 코드를 추가해야 하는 경우가 발생합니다. 하지만 현재 작업 중인 코드 역시 수정이 끝난 것은 아닙니다.

이러한 상황은 실제 개발 작업 과정에서 자주 발생합니다. 일부 커밋하지 않은 작업 내용이 남아 있다면 브랜치 간에 이동할 수 없습니다. 그럼 어떻게 현재 수정 작업을 멈추고, 다른 브랜치에 있는 코드를 수정할 수 있을까요? 이러한 상황에서는 스태시(stash) 기능을 사용할 수 있습니다. 스태시는 간단히 말해 '안전한 보관'입니다.

▼ 그림 7-1 스태시



깃은 완료되지 않은 작업(커밋되지 않은 변경 내용)이 남아 있을 때, 현재 작업을 임시로 저장할 수 있는 스태시 기능을 제공합니다. 스태시는 '현재 워킹 디렉터리 내역을 별도의 스택 영역에 잠시 보관하라'는 명령입니다. 스태시는 브랜치를 이동할 때 작업 중인 내용 때문에 워킹 디렉터리가 충돌하는 것을 방지하는 데 사용합니다.

스태시 명령을 실행하면 현재 작업 중인 내용은 임시 저장되고, 수정 전 마지막 커밋 상태로 돌아갑니다. 즉, 이전 커밋 후 작업하지 않은 상태의 워킹 디렉터리가 됩니다.

7.1.1 기존 작업 도중에 새로운 변경 요청

개발자는 다양한 상황과 마주칩니다. 대표적으로 수정 작업을 하는 도중에 또 다른 수정 요청이 있는 경우입니다. 이때 현재 작업은 잠시 멈추고, 또 다른 요청을 반영하려면 새로운 브랜치가 필요합니다.

상황 1

실습을 위해 이야기한 상황과 유사한 작업 환경을 만들어 보겠습니다. 새로운 작업을 할 수 있게 브랜치를 하나 생성합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy07 (main)


$ git checkout -b feature 브랜치 수정과 체크아웃 동시에 처리


Switched to a new branch 'feature'
```

생성된 feature 브랜치에서 stash.htm 파일을 수정하고, 저장한 후 커밋합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy07 (feature)


$ code stash.htm VS Code 실행
```

stash.htm

```
<h1>스태시를 실습합니다.</h1>


<h2>새로운 기능을 시작합니다.</h2>
```

```
infoh@DESKTOP MINGW64 /e/gitstudy07 (feature)


$ git commit -am "new feature start" 등록 및 커밋


[feature a43043e] new feature start


 1 file changed, 2 insertions(+), 1 deletion(-)
```

7.1.2 새 코드 작성 중 기존 코드를 수정

앞의 상황에 이어 또 다른 상황을 고려해 보겠습니다.

상황 2

기존 코드를 수정하는 도중에 또 다른 수정 요청이 들어옵니다. 기존 코드에 버그가 발견되어 요청된 긴급 수정 건입니다. 작업 중인 브랜치는 잠시 멈추고, 버그를 수정하려면 다른 브랜치에서 작업해야 합니다. 즉, 두 브랜치 작업을 동시에 수행해야 합니다.

7.1.1절 실습에 이어 진행해 보겠습니다.

현재 브랜치는 작업 중인 코드로 불완전합니다. 새롭게 발견된 버그를 수정하려면 또 다른 원본 브랜치가 필요합니다. 원본 main 브랜치에서 새로운 브랜치를 생성해야 합니다. 따라서 main 브랜치로 체크아웃합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy07 (feature)


$ git checkout main 브랜치 이동


error: Your local changes to the following files would be overwritten by checkout:


        stash.htm


Please commit your changes or stash them before you switch branches.


Aborting 오류 메시지
```

앗, 원본 main 브랜치로 체크아웃되지 않습니다! 오류 메시지만 출력됩니다. 오류 메시지 내용은 워킹 디렉터리에 커밋하지 않은 작업이 남아 있어 현재 브랜치를 변경할 수 없다는 것입니다.

이러한 오류는 현재 수정하는 작업들이 체크아웃으로 다른 브랜치의 워킹 디렉터리에 영향을 줄 수 있기 때문입니다. 깃은 이러한 충돌을 미연에 방지하고자 작업 중인 내용이 있을 때는 브랜치 간 이동을 제한합니다.

브랜치를 변경하기 위한 체크아웃을 하려면 현재 작업 중인 워킹 디렉터리와 스테이지를 깔끔히 정리해야 합니다. 하지만 아직은 새로운 기능의 작업이 완료되지 않아 커밋할 수 없습니다. 이때 스태시를 활용하면 매우 유용합니다.

7.1.3 스태시의 임시 스택 영역에 작업 중인 코드 저장

우리는 현재 작업 중인 브랜치의 워킹 디렉터리를 정리하지 못한 상태입니다. 따라서 현재 브랜치에서 버그를 수정하기 위한 다른 브랜치로 체크아웃할 수 없습니다.

다시 status 명령어를 실행하여 상태를 확인합시다.

```
infoh@DESKTOP MINGW64 /e/gitstudy07 (feature)


$ git status 상태 확인


On branch feature


Changes not staged for commit:


  (use "git add <file>..." to update what will be committed)


  (use "git checkout -- <file>..." to discard changes in working directory)


        modified:   stash.htm 수정 상태


no changes added to commit (use "git add" and/or "git commit -a")
```

브랜치를 이동할 수 없는 이유는 코드가 아직 수정 중인 상태이기 때문입니다. 이를 해결해야 다른 브랜치로 이동할 수 있습니다.

스태시 명령어는 수정 중인 내역을 커밋하지 않고도 브랜치를 이동할 수 있게 워킹 디렉터리를 깨끗이 청소합니다. 따라서 커밋 대신 스태시 명령을 실행하면 됩니다. 스태시는 영구적인 커밋 기록 대신 현재 작업들을 임시 스택 영역에 저장합니다. 간략하게는 git stash 명령어만 사용합니다.

$ git stash

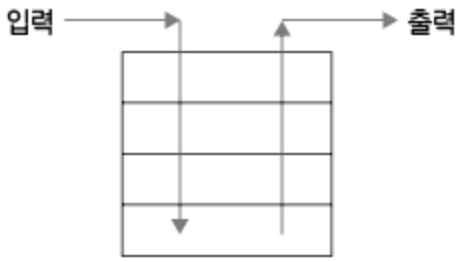
또는 save 명령어를 추가하여 사용합니다. git stash save 명령어는 스태시 여러 개를 생성할 때 유용합니다.

```
$ git stash save
```

7.1.4 임시 저장 영역의 스택 목록

스태시의 저장 영역은 스택 구조입니다. 스택은 FILO(First Input Last Out) 구조로, 마지막에 입력한 자료가 제일 먼저 출력되는 데이터 저장 방식입니다.

▼ 그림 7-2 스택은 FILO 구조



스태시에 저장된 스택은 list 옵션을 사용하여 확인할 수 있습니다.

$ git stash list

깃은 스태시된 객체들을 .git/refs/stash에 저장합니다. 다음 예처럼 .git/refs/stash에 저장된 스태시 객체에서 cat 명령어로 내용을 확인할 수 있습니다.

예

```
$ cat .git/refs/stash


c5be53ff322fbc8f0d06b800a2b3637aa385cb62
```

main 브랜치에서 git stash list 명령어를 실행하여 목록을 확인합시다.

```
infoh@DESKTOP MINGW64 /e/gitstudy07 (main)


$ git stash list 스태시 목록


stash@{0}: WIP on main: a43043e first commit
```

7.1.5 임시 저장한 스태시 불러오기

스태시를 사용하는 목적은 현재 워킹 디렉터리를 커밋하지 않고 임시로 저장하기 위해서입니다. 임시 저장된 내용은 다른 수정 작업을 완료한 후에 다시 불러와 사용할 수 있습니다. 앞에서 임시 저장한 스태시를 불러와 실습을 계속 진행하겠습니다. 먼저 main 브랜치의 파일 버그를 수정한 후 커밋하겠습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy07 (main)


$ code stash.htm VS Code 실행
```

stash.htm

```
<h1>stash를 실습합니다.</h1>
```

한글 '스태시'를 영문 'stash'로 변경했습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy07 (main)


$ git commit -am "bug fix main" 등록 및 커밋


[main 535d22c] bug fix main


 1 file changed, 1 insertion(+), 1 deletion(-)
```

버그를 수정했습니다. 다시 원래 브랜치(feature)로 이동합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy07 (main)

$ git checkout feature 브랜치 이동

Switched to branch 'feature'



infoh@DESKTOP MINGW64 /e/gitstudy07 (feature)
```

7.1.6 스태시 복원으로 충돌

스태시를 복원할 때 워킹 디렉터리의 상태는 깨끗해야 합니다. 스택에 저장된 스태시 내용이 다시 워킹 디렉터리로 복구될 때, 수정된 작업 내용과 현재 워킹 디렉터리를 병합하기 때문입니다. 복구되는 브랜치의 워킹 디렉터리가 깨끗하지 않다면 병합 과정에서 충돌이 발생할 가능성이 많습니다.

특히 스태시를 복원할 때 같은 파일에서 동일한 부분을 변경했다면 즉시 충돌이 발생합니다. 스태시를 복원할 때 충돌이 생기면 직접 문제를 해결해야 합니다. 복원하는 도중 충돌이 생기면 스태시는 스택에 저장된 내용을 자동으로 삭제하지 않습니다. 직접 충돌을 해결한 후 스태시 목록을 수동으로 삭제해야 합니다. 스태시 충돌이 예상된다면 스태시용 브랜치를 하나 생성해서 작업하는 것을 추천합니다.

다음은 새로운 브랜치를 생성한 후 스태시를 적용하는 명령입니다. 스태시 스택에 저장된 내용으로 새로운 브랜치를 동시에 생성할 수 있습니다.

$ git stash branch 브랜치이름

스태시로 이전 내용을 복원한 상태입니다. 아직 커밋을 수행하지 않았기 때문에 작업 중인 내용은 수정된 상태로 워킹 디렉터리에 남아 있습니다. 실습을 위해 현재의 임시 작업 내용을 다시 스태시해 보겠습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy07 (feature)


$ git stash 스태시 저장


Saved working directory and index state WIP on feature: a43043e new feature start
```

7.1.7 스태시 복사

스태시는 브랜치 작업들을 임시로 저장할 때 사용합니다. 또 임시 저장된 작업을 스태시 명령 이전의 브랜치 상태로 되돌려 놓습니다. 스태시를 사용한 저장과 복원은 서로 다른 브랜치에도 가능합니다. 반드시 이전에 실행한 브랜치와 같은 브랜치에서 할 필요는 없습니다.

앞에서 실습한 것처럼 다른 브랜치에서 스태시를 실행한 후 새로운 test 브랜치를 생성하여 스태시를 복원했습니다. 스태시 스택에 저장된 항목들은 어느 브랜치에서나 복원이 가능합니다. apply 옵션은 스택에 저장된 항목을 불러와 현재 브랜치로 복원합니다.

$ git stash apply

스태시 복원은 pop, apply 명령어 2개를 제공합니다. 두 명령어에는 차이가 있습니다. 스태시의 pop 명령어는 스택 내용을 복원한 후 스택 목록에서 자동으로 삭제합니다. 즉, pop 명령어는 스택 내용을 워킹 디렉터리로 이동하는 것과 같습니다. 하지만 스태시 복원을 하고 난 후 스택 목록을 삭제하고 싶지 않을 때도 있습니다. 이때 apply 옵션을 사용합니다. 스택 목록을 읽은 후 자동으로 삭제하지 않기 때문에 반복적으로 스택에서 스태시 내용을 읽어 올 수 있습니다. apply 명령어는 스태시 내용을 워킹 디렉터리로 복사하는 것과 같습니다.

7.1.8 스태시 삭제

스태시는 복원할 때 한 번 호출하면 자동으로 스택에서 삭제됩니다. 하지만 충돌이 발생하거나 apply 명령어로 워킹 디렉터리를 복구할 때는 스택에서 자동으로 삭제되지 않습니다. 이때는 별도 명령을 실행해야만 스태시 목록에서 삭제됩니다.

$ git stash drop

이전 실습은 test 브랜치에서 저장한 임시 작업 내용을 apply 명령어를 사용하여 feature 브랜치로 복원했습니다. 그리고 스택 목록은 삭제되지 않고 남아 있습니다. 남아 있는 스택 목록을 삭제해 보겠습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy07 (feature)


$ git stash list 스태시 목록


stash@{0}: WIP on test: a43043e new feature start




infoh@DESKTOP MINGW64 /e/gitstudy07 (feature)


$ git stash drop 스태시 삭제


Dropped refs/stash@{0} (6f494132548e24cf44660a60739df21f78be587b)
```

참고로 스태시를 너무 많이 만들어 사용하는 것은 바람직하지 않습니다. 사용한 스태시는 그때그때 삭제하여 정리하는 것이 좋습니다.

7.1.9 소스트리에서 스태시 사용

먼저 소스트리의 새 탭에서 Add 버튼을 클릭합니다. 탐색을 눌러 앞에서 만든 gitstudy07 폴더를 찾아 선택한 후 추가를 누릅니다. 그러면 gitstudy07 저장소와 연결됩니다.

소스트리에서는 코드 파일을 수정한 후 커밋하지 않은 내용이 있는 상태가 되면 자동으로 스태시



버튼이 활성화됩니다. 다음과 같이 왼쪽에서 feature 브랜치를 선택하면 아래쪽에서 자세한 설명을 볼 수 있습니다. 스태시 버튼을 클릭합니다.

▼ 그림 7-3 소스트리에서의 스태시 버튼



7.2 워킹 디렉터리 청소

개발하는 과정에서는 컴파일 등 임시로 생성되는 파일들이 생깁니다. 이 파일들은 .gitignore 목록에 넣어 관리할 수도 있지만, 반복해서 불필요하게 생성되는 파일들은 귀찮을 뿐입니다. 이때 clean 명령어를 사용하면 워킹 디렉터리에 있는 추적되지 않는 파일들을 찾아 삭제합니다. clean 명령어를 사용할 때는 주의할 점이 있는데, clean 명령어를 실행하는 순간 워킹 디렉터리의 추적되지 않는 모든 파일을 삭제한다는 것입니다.

clean 명령어는 단독으로 실행하거나 몇 가지 추가 옵션을 같이 사용할 수 있습니다.

$ git clean

실습을 위해 feature 브랜치에 새 파일을 하나 생성합니다.

infoh@DESKTOP MINGW64 /e/gitstudy07 (feature)

$ echo "this is temp" > temp.htm 새로운 파일

infoh@DESKTOP MINGW64 /e/gitstudy07 (feature)2

$ git status 상태 확인

On branch feature

Changes not staged for commit:

  (use "git add <file>..." to update what will be committed)

  (use "git restore <file>..." to discard changes in working directory)

  modified: stash.htm

Untracked files:

  (use "git add <file>..." to include in what will be committed)

  temp.htm 추적하지 않는 파일

```
no changes added to commit (use "git add" and/or "git commit -a")
```

7.3 정리

깃을 안정적으로 잘 사용하려면 워킹 디렉터리를 항상 깨끗하게 유지 관리해야 합니다. 워킹 디렉터리는 코드를 생성하고 수정하는 모든 작업 공간입니다. 워킹 디렉터리가 정리되어 있지 않으면, 브랜치를 변경하여 다른 작업을 할 수 없습니다.

워킹 디렉터리를 정리하는 방법은 수정된 코드를 스테이지하고, 최종 스냅샷을 생성하여 커밋하는 것입니다. 커밋하지 못할 때는 이 장에서 배운 스태시를 적용하여 작업 내용을 임시로 저장한 후 사용해야 합니다.

1 WIP는 Work In Progress의 약어입니다. 일반적으로 진행 중인 작업이라는 의미입니다.

2 7.1.9절 소스트리에서 스태시 실습을 했다면 “Changes not staged for commit ~ modified: stash.htm” 메시지는 출력되지 않습니다.